Retrieval Augmented Generation

Advanced Retrieval Augmented Generation

Retrieval augmented generation, or RAG, is an architecture for optimizing the performance of an artificial intelligence (AI) model by connecting it with external knowledge bases.

RAG helps large language models (LLMs) deliver more relevant responses at a higher quality.

Generative AI (gen AI) models are trained on large datasets and refer to this information to generate outputs.

However, training datasets are finite and limited to the information the AI developer can access—public domain works, internet articles, social media content and other publicly accessible data.

RAG allows generative AI models to access additional external knowledge bases, such as internal organizational data, scholarly journals and specialized datasets.

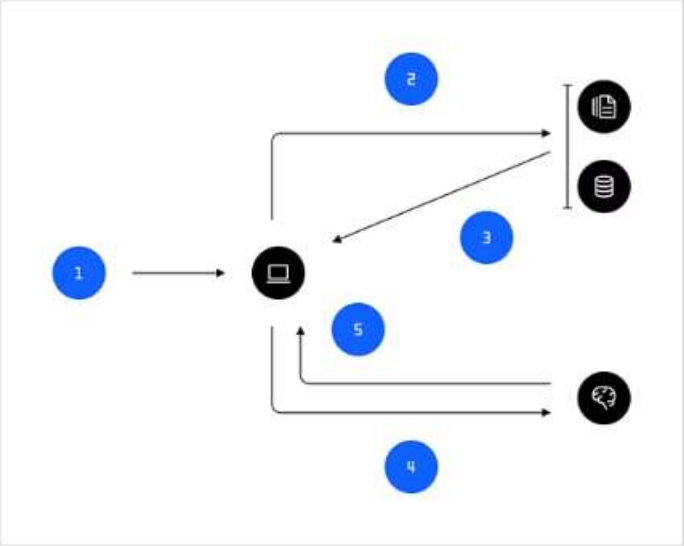
By integrating relevant information into the generation process, chatbots and other natural language processing (NLP) tools can create more accurate domain-specific content without needing further training.

How does RAG work?

RAG works by combining information retrieval models with generative AI models to produce more authoritative content. RAG systems query a knowledge base and add more context to a user prompt before generating a response.

Standard LLMs source information from their training datasets. RAG adds an information retrieval component to the AI workflow, gathering relevant information and feeding that to the generative AI model to enhance response quality and utility.

RAG systems follow a five-stage process:



1.The user submits a prompt.

2.The information **retrieval** model queries the knowledge base for relevant data.

3.Relevant information is returned from the knowledge base to the integration layer.

4.The RAG system engineers an **augmented** prompt to the LLM with enhanced context from the retrieved data.

5.The LLM **generates** an output and returns an output to the user.

This process showcases how RAG gets its name. The RAG system *retrieves* data from the knowledge base, *augments* the prompt with added context and *generates* a response.

Components of a RAG system

RAG systems contain four primary components:

•The **knowledge base**: The external data repository for the system.

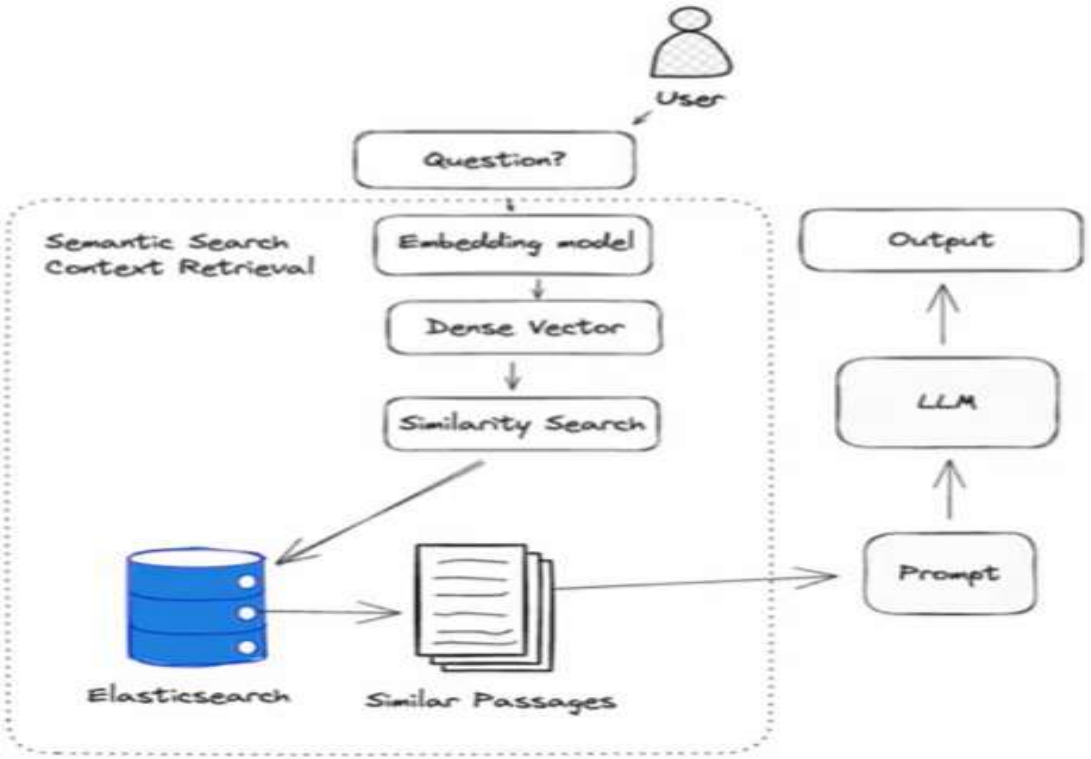
•The **retriever**: An AI model that searches the knowledge base for relevant data.

•The **integration layer**: The portion of the RAG architecture that coordinates its overall functioning.

•The **generator**: A generative AI model that creates an output based on the user query and retrieved data.

Other components might include a *ranker*, which ranks retrieved data based on relevance, and an *output handler*, which formats the generated response for the user.

Building RAG is Easy



RAG use cases

RAG systems essentially enable users to query databases with conversational language. The data-powered question-answering abilities of RAG systems have been applied across a range of use cases, including:

- Specialized chatbots and virtual assistants
- Research
- Content generation
- Market analysis and product development
- Knowledge engines
- Recommendation services

RAG Breaks the Old Rules

Summarize

In RAG, the retriever isn't helping a user — it's helping a model. That changes everything.

Traditional IR	RAG Systems
User browses ranked list	LLM consumes retrieved content
Optimize for top-k relevance (MAP, NDCG)	Optimize for grounding and synthesis
Queries are short, keyword-based	Queries are natural, long-form, ambiguous
Focus on document ranking	Focus on passage evidence selection
Evaluation: clicks, rank positions	Evaluation: faithfulness, hallucination rate
Retrieval ends with the user	Retrieval fuels generation

Antonio Mallia

Basic RAG Systems

Pros	Cons
Useful in simple contexts	Hallucination issues
Quick information retrieval	Lack of domain specificity
	Struggles with complex queries

Roadmap

- Problem with RAG today
- How to scope out RAG
- Retrieval
  - BM25
  - Language Models
  - Agentic Search/RAG

Retrieval Enhancement Techniques

The Evolution of Retrieval

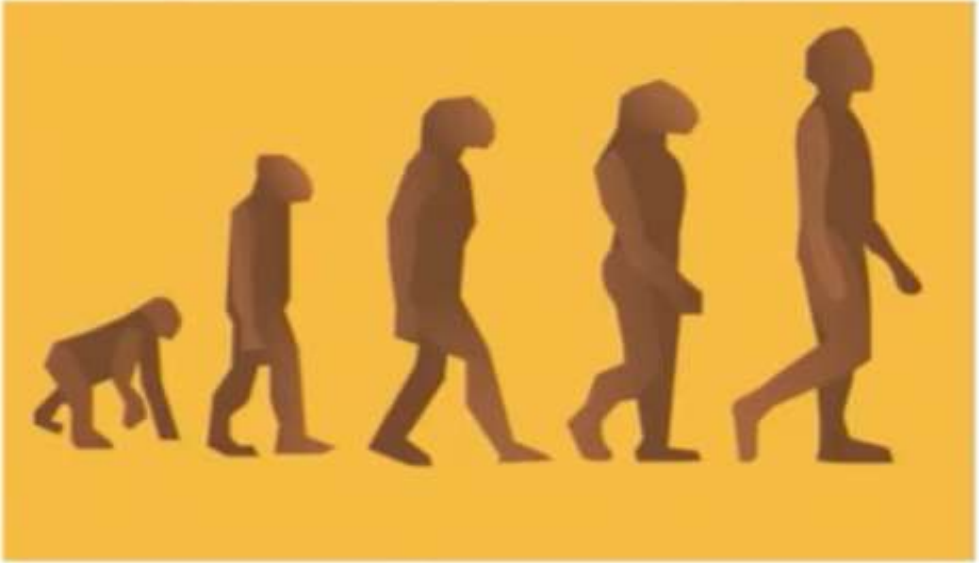
From Ranked Lists to Generative Evidence

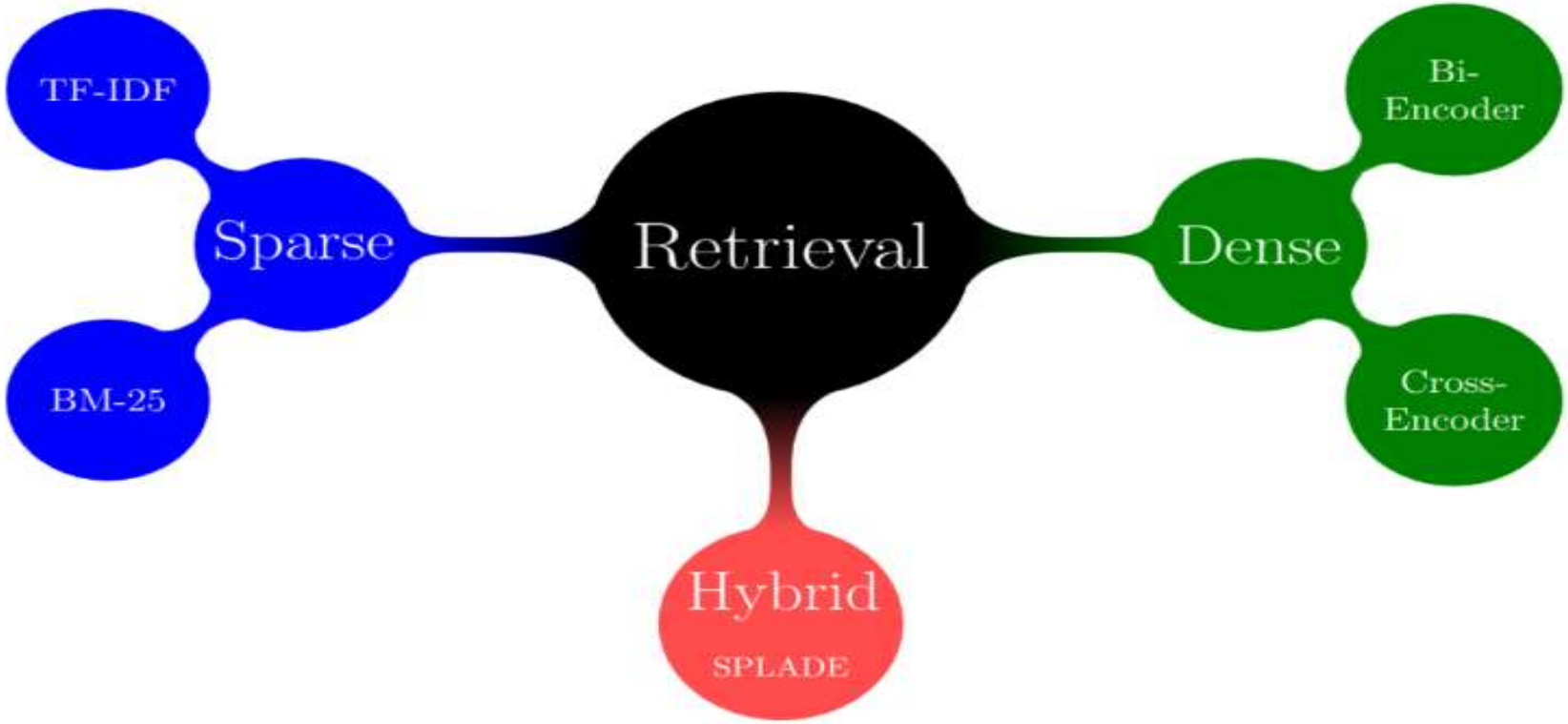
Traditional IR: Ranked lists optimized for precision, recall, and user clicks.

RAG: Needs evidence retrieval that supports generation, not navigation.

What matters now:

- Faithfulness, coverage, reasoning chains
- Retrieval that supports answer synthesis, not just result finding





Information retrieval based on sparse, dense, and hybrid text representations

Introduction

- Information retrieval is the task of finding documents that satisfy an information need from a large collection of documents.

- Given the vast amount of data, efficient information retrieval techniques are essential to a number of applications from web search to recommendations to conversational chatbots.

- In a typical retrieval system, the information need is represented by a user query.

- The user query is efficiently compared to all the available documents and the most relevant documents are returned to the user.

- A crucial task in information retrieval is the representation of the documents and the query. There are two major representations - sparse and dense.

Sparse Representations

Sparse retrieval methods represent text by the counts of each word in the vocabulary.

Since the vocabulary is generally very large and documents typically contain only a small subset of the vocabulary, most of the counts are zero, resulting in a sparse vector.

A major advantage of sparse representation is the efficiency of retrieval using an inverted index.

An inverted index keeps track of the documents containing each word or phrase.

Given a query, an inverted index can quickly retrieve all documents containing the query terms.

The retrieved documents can then be scored based on the frequency of the query terms. However, not all terms are equally important, and their significance varies.

Term Frequency - Inverse Document Frequency (TF-IDF)

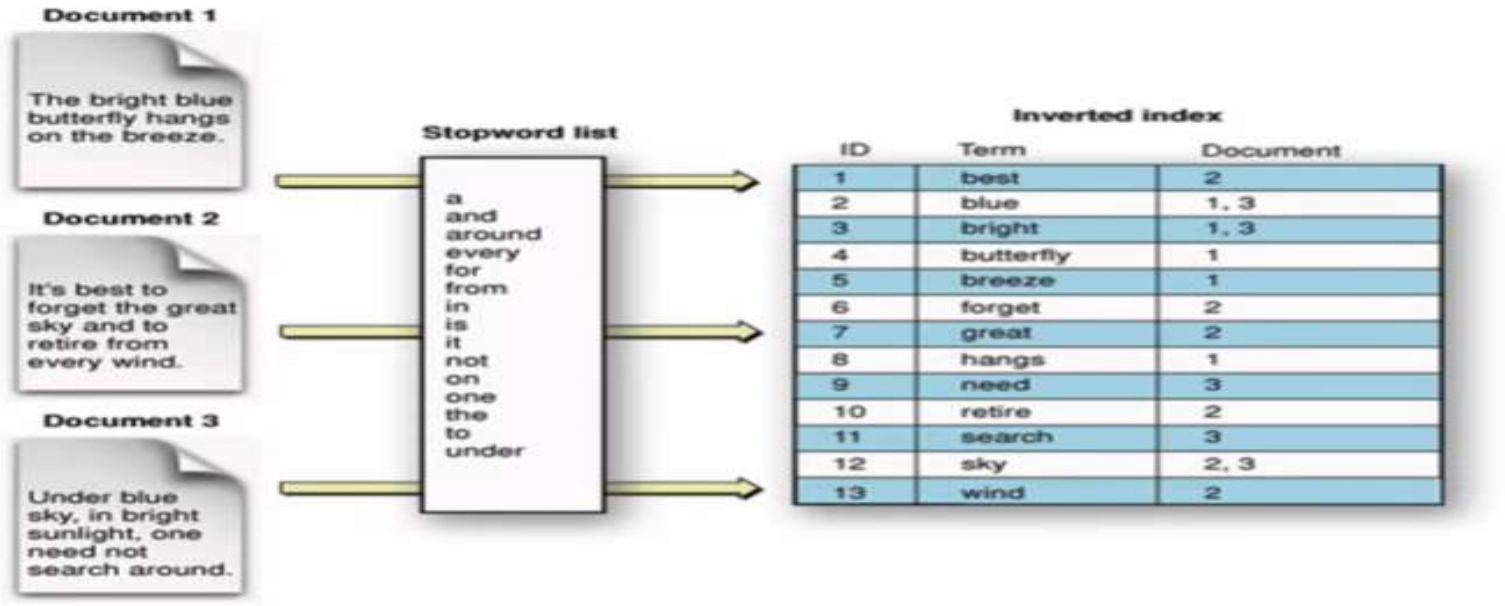
Some words appear in nearly all documents, diminishing their utility for distinguishing relevant documents. To improve retriever performance, TF-IDF assigns weights to terms based on their frequency across documents, counting the words while attenuating the influence of terms with high document frequency.

BM-25

A large document will inherently have a higher TF-IDF similarity with a query than a smaller one, potentially skewing results. BM-25 addresses this by incorporating document length normalization into TF-IDF, adjusting term weights to ensure fairer comparisons. BM-25 is a robust baseline retrieval method, performing competitively across various benchmarks.

Sparse Retrieval (BM25),

BM25



ID	Term	Document
1	best	2
2	blue	1, 3
3	bright	1, 3
4	butterfly	1
5	breeze	1
6	forget	2
7	great	2
8	hangs	1
9	need	3
10	retire	2
11	search	3
12	sky	2, 3
13	wind	2

BM25 is a refinement of TF-IDF.

As you can see with TF-IDF, the term frequency tends to grow as a document gets longer, which means very long documents can receive disproportionately large scores.

BM25 corrects for this by adjusting the score using both the document length and the average document length, so long documents don’t get unfairly high scores.

Another issue with plain TF-IDF is that a term’s contribution increases linearly with each repetition in a document (e.g. “APT APT APT APT …”).

BM25 counteracts this by saturating the term-frequency effect—beyond a certain point, repeating a word adds little additional importance (a smoothing effect).

Incorporating these two ideas, the BM25 scoring function is:

$$\mathrm{BM25}(D, Q) = \sum_{q_i \in Q} \mathrm{IDF}(q_i) \cdot \frac{f(q_i, D)\,(k_1 + 1)}{f(q_i, D) + k_1 \left(1 - b + b\,\frac{|D|}{\mathrm{avgdl}}\right)}$$

- The left factor is BM25's IDF, and the right fraction is BM25's TF component.

Dense Representations

- Sparse retrieval methods are efficient but suffer from the vocabulary mismatch problem—lexical terms present in the query but not in a relevant document, and vice versa.

- Dense retrieval methods alleviate this problem by representing text in continuous vector spaces, allowing for better semantic matching even when exact terms differ.

- Most of the common dense representations are based on some form of the transformer architecture.

- A general distinction between the types of dense representations is based on whether the collection of documents can be encoded independent of the query or not.

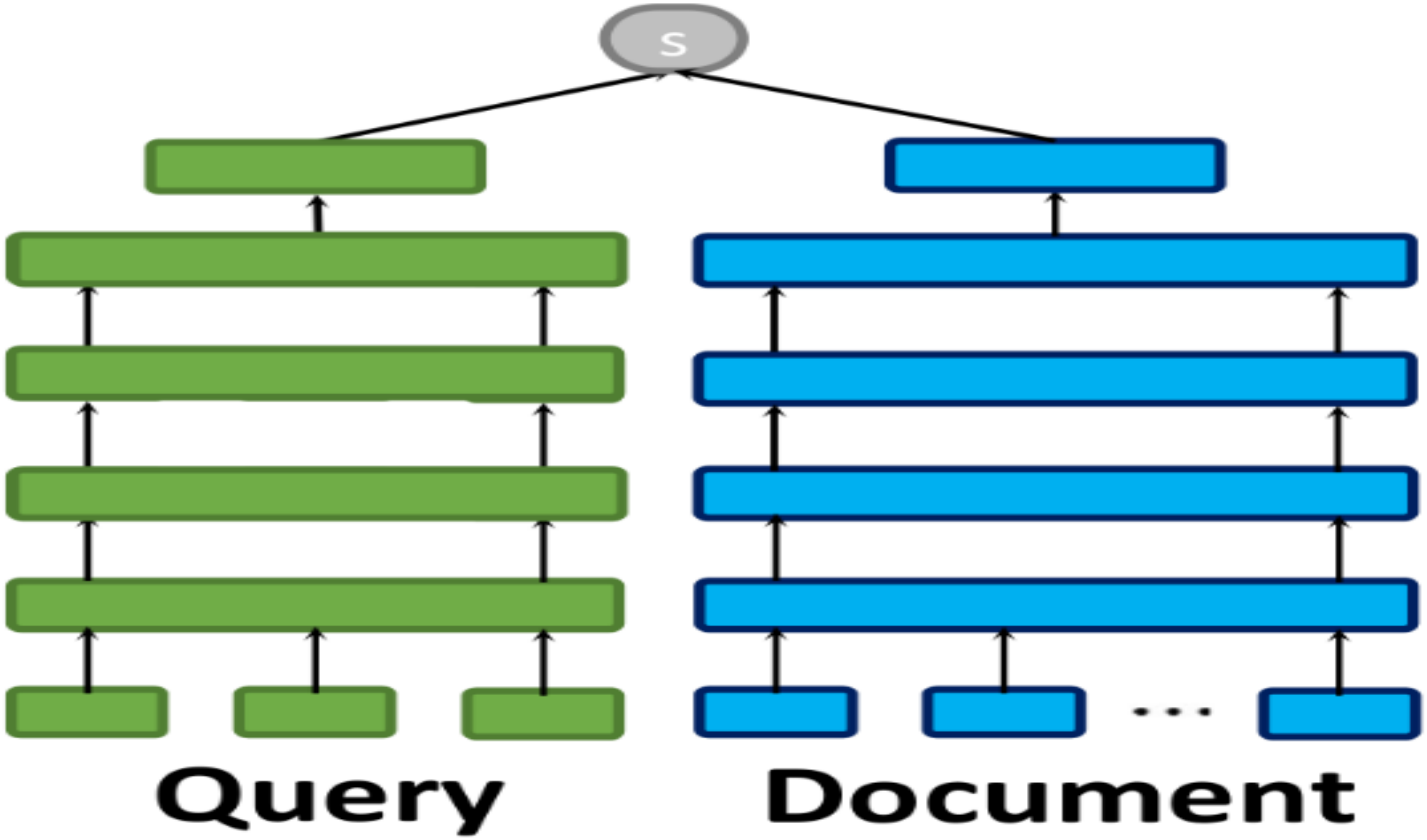
Bi-Encoders

Usually, the collection of documents from which we retrieve relevant documents is quite large.

Bi-Encoder-based representations encode the documents independently of the queries.

Since encoding with a transformer is expensive, documents are encoded once and stored in a database.

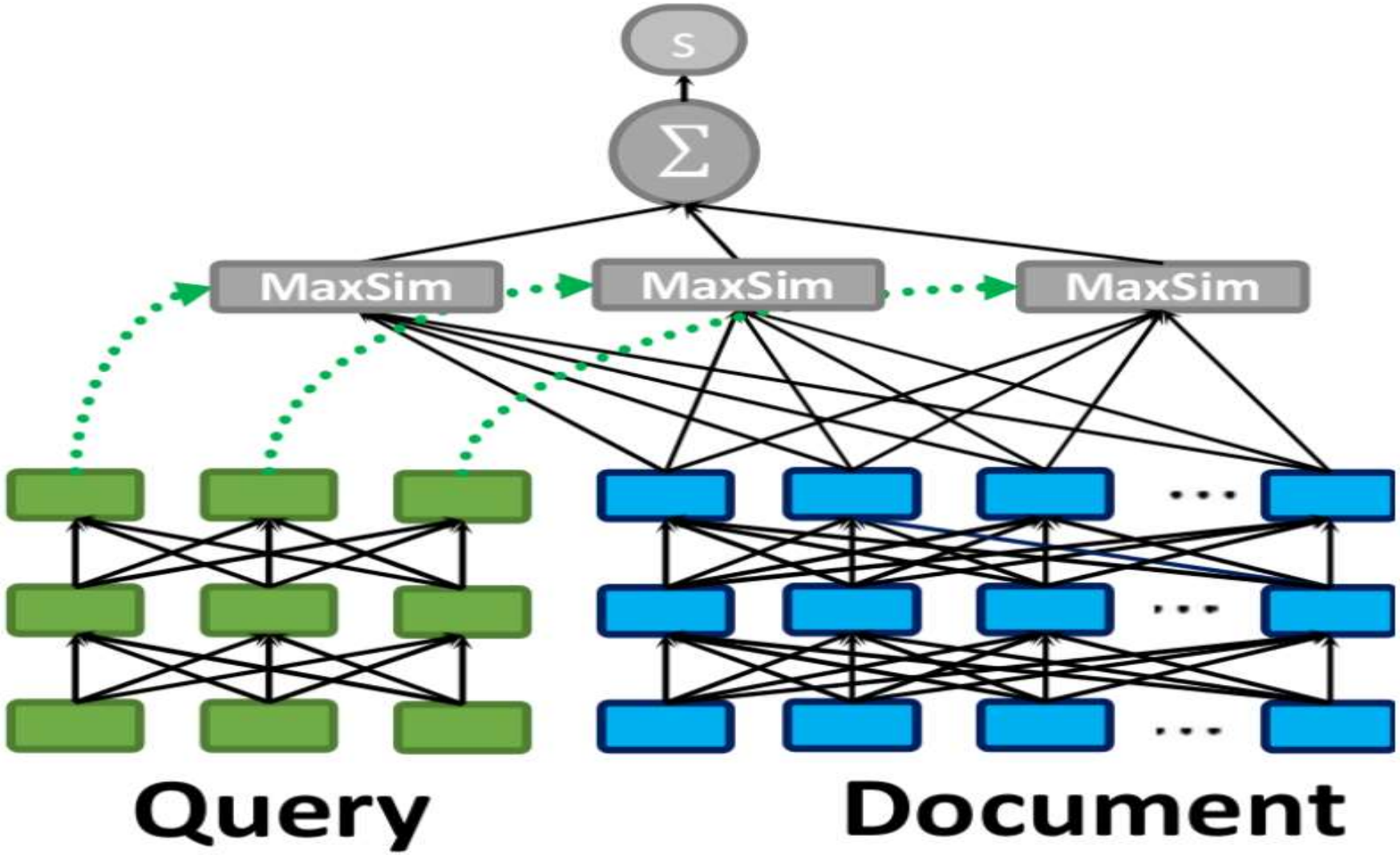
During a search, the query is encoded, and an approximate nearest neighbour search is performed to find the most relevant documents based on cosine similarity between the query vector and the document vectors.



Bi-Encoder retrieval system - the documents are encoded once for all queries. Figure source [1]

The majority of stage-1 retrieval systems use the bi-encoder architecture. Some of the common bi-encoder systems are given below.

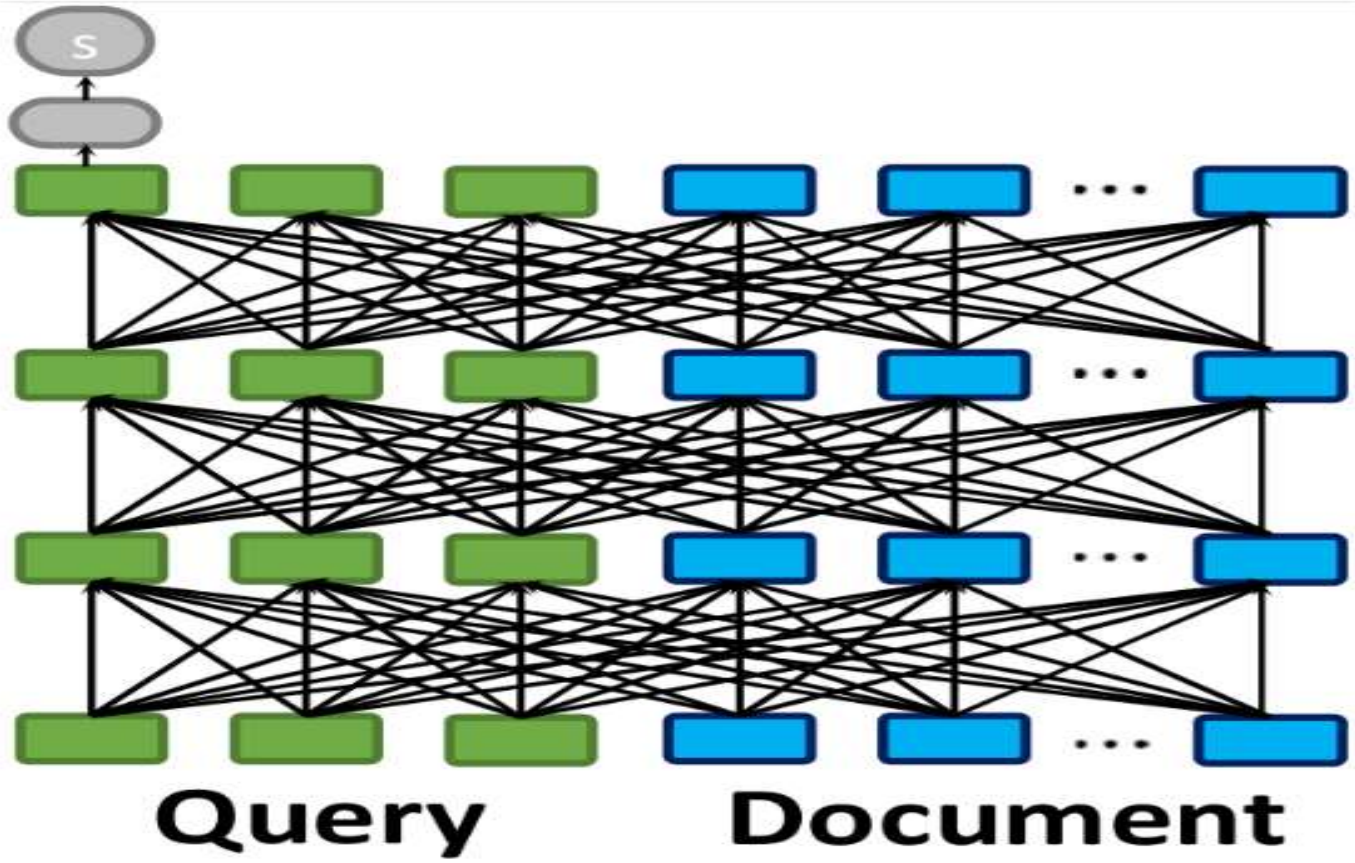
- **Dense Passage Retrieval** (DPR) and **Sentence BERT** (SBERT): Both of these techniques use a supervised training set of a (query, positive document, negative document) to contrastively train a transformer encoder model [2].
- **Contriever**: Similar to the above techniques contriever uses contrastive training. However, contriever uses self-supervised learning, i.e., it generates the (query, positive document, negative document) tuple from general text. This allows contriever to train on much larger datasets and thus performing better than supervised techniques [3].
- **ColBERT (Contextualized Late Interaction)**: The previous techniques encode the document tokens and pool the resulting embeddings to generate a single embedding vector. ColBERT, in contrast, encodes the documents with a pre-trained BERT and keep the embeddings of all the tokens. During search, the query tokens are embedded and a score is calculated using a MaxSim operation involving all tokens from both the document and the query.



ColBERT stores the embeddings of all the document tokens and perform late interaction with query at search time. Figure source [1]

Cross-Encoder

A cross-encoder architecture encodes both the document and the query using a single encoder simultaneously to get the similarity score between the query and each document. Since this process is very expensive, cross-encoders are used as stage 2 re-rankers where the number of documents have been reduced to a few dozens instead of hundreds of millions.



Common cross-encoders include the following

•**BERT**: In a pre-trained BERT can directly be used as a cross-encoder, feeding both the query and each document seperated by the [SEP] token and using the embedding of [CLS] token to compute the similarity score.

•**T5**: Similar to BERT, T5 can be used to concatenate the query and each document to compute the score of each document for retrieval.

Hybrid Techniques

Sparse techniques are efficient, and dense techniques alleviate the vocabulary mismatch problem. Hybrid techniques attempt to unify the best of both worlds using document expansion.

One of the most popular hybrid techniques is the SParse Lexical AnD Expansion (SPLADE) model [4].

Hybrid techniques retain the efficiency of sparse methods by focusing on exact term matches while using dense embeddings to identify related terms for expansion.

In SPLADE, a sparse representation of the document is expanded with additional related terms identified through dense embeddings, addressing the vocabulary mismatch problem.

For example, consider a one-hot vector for the query "car":

$q = [0,1,0,0, \ldots ,0]$

Here, the second position corresponds to "car," and all other positions are zero. SPLADE uses dense embeddings to expand the document representation to include related terms like "vehicle" and "automobile," resulting in:

$q = [0,1,0,1, \ldots ,1]$

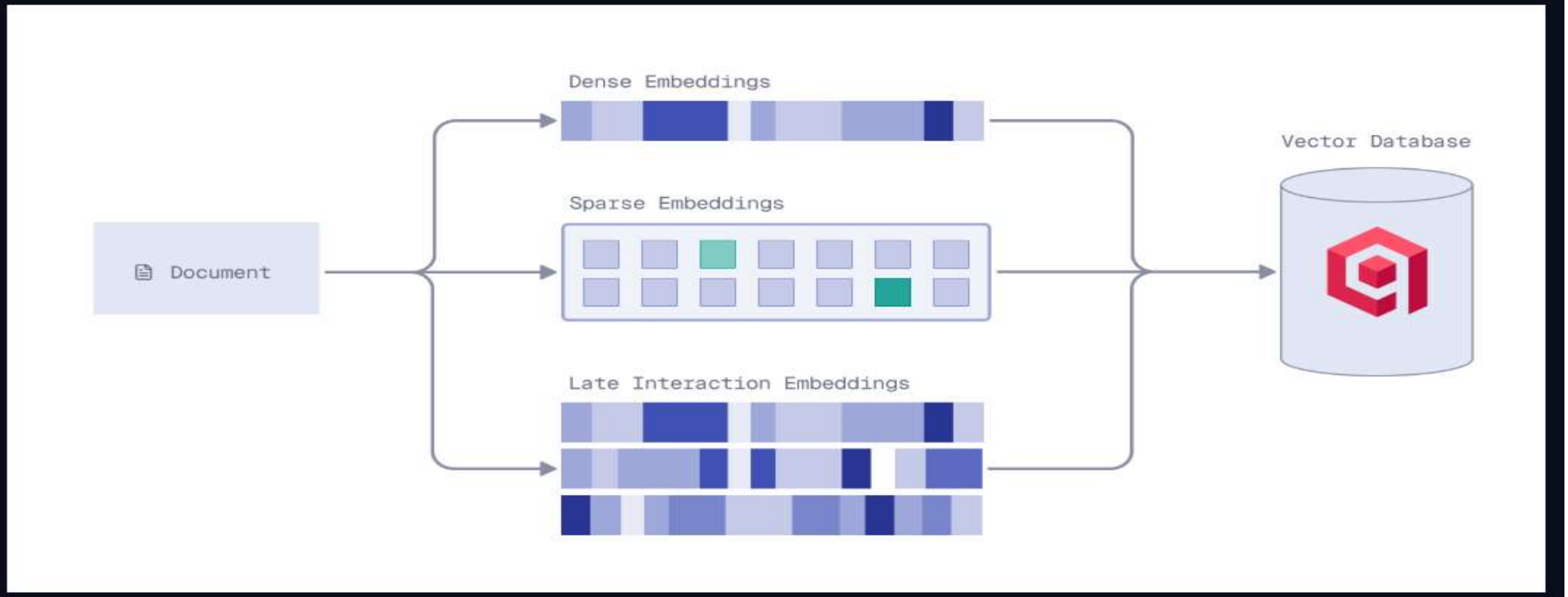
This ensures the query matches documents containing these related terms, improving recall and relevance.

Reranking Hybrid Search Results with Qdrant Vector Database

Hybrid search combines dense and sparse retrieval to deliver precise and comprehensive results. By adding reranking with ColBERT, you can further refine search outputs for maximum relevance.

In this guide, we’ll show you how to implement hybrid search with reranking in Qdrant, leveraging dense, sparse, and late interaction embeddings to create an efficient, high-accuracy search system. Let’s get started!

Let's start by breaking down the architecture:

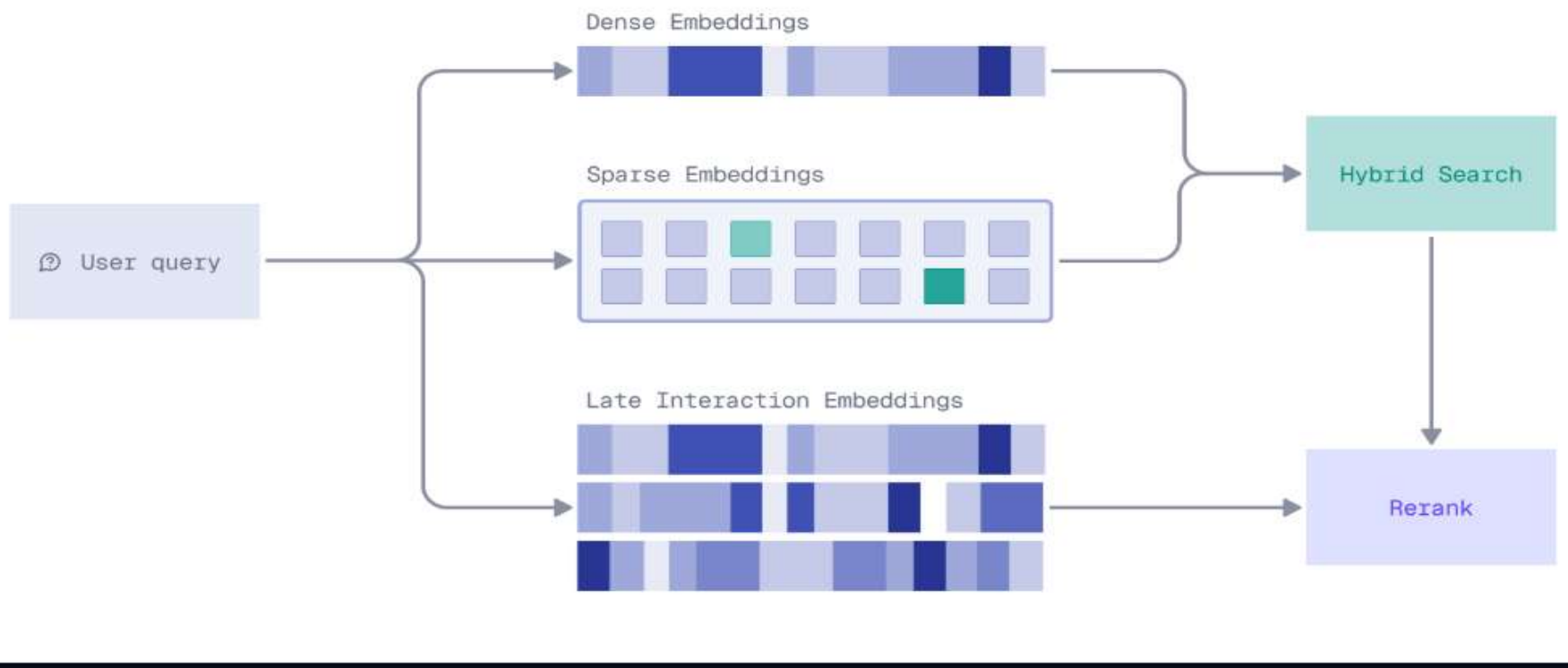


Processing Dense, Sparse, and Late Interaction Embeddings in Vector Databases (VDB)

Ingestion Stage

Here’s how we’re going to set up the advanced hybrid search. The process is similar to what we did earlier but with a few powerful additions:

1. **Documents**: Just like before, we start with the raw input—our set of documents that need to be indexed for search.
2. **Dense Embeddings**: We’ll generate dense embeddings for each document, just like in the basic search. These embeddings capture the deeper, semantic meanings behind the text.
3. **Sparse Embeddings**: This is where it gets interesting. Alongside dense embeddings, we’ll create sparse embeddings using more traditional, keyword-based methods. Specifically, we’ll use BM25, a probabilistic retrieval model. BM25 ranks documents based on how relevant their terms are to a given query, taking into account how often terms appear, document length, and how common the term is across all documents. It’s perfect for keyword-heavy searches.
4. **Late Interaction Embeddings**: Now, we add the magic of ColBERT. ColBERT uses a two-stage approach. First, it generates contextualized embeddings for both queries and documents using BERT, and then it performs late interaction—matching those embeddings efficiently using a dot product to fine-tune relevance. This step allows for deeper, contextual understanding, making sure you get the most precise results.
5. **Vector Database**: All of these embeddings—dense, sparse, and late interaction—are stored in a vector database like Qdrant. This allows you to efficiently search, retrieve, and rerank your documents based on multiple layers of relevance.



Query Retrieval and Reranking Process in Search Systems

Retrieval Stage

Now, let's talk about how we're going to pull the best results once the user submits a query:

1. **User's Query**: The user enters a query, and that query is transformed into multiple types of embeddings. We're talking about representations that capture both the deeper meaning (dense) and specific keywords (sparse).
2. **Embeddings**: The query gets converted into various embeddings—some for understanding the semantics (dense embeddings) and others for focusing on keyword matches (sparse embeddings).
3. **Hybrid Search**: Our hybrid search uses both dense and sparse embeddings to find the most relevant documents. The dense embeddings ensure we capture the overall meaning of the query, while sparse embeddings make sure we don't miss out on those key, important terms.
4. **Rerank**: Once we've got a set of documents, the final step is reranking. This is where late interaction embeddings come into play, giving you results that are not only relevant but tuned to your query by prioritizing the documents that truly meet the user's intent.

Refining the Search: Query Transformation

User queries can be ambiguous, overly concise, or lack sufficient context for effective semantic search. Query transformation techniques aim to modify the original user query into one or more optimized queries that are more likely to yield relevant documents from the vector store.

Query Expansion

The simplest form involves expanding the query with related terms or concepts, often using an LLM. The goal is to broaden the search aperture slightly to catch documents that might use different terminology for the same idea.

For example, a user query like "RAG performance issues" could be expanded by an LLM to include terms like "Retrieval-Augmented Generation latency", "vector search optimization", "RAG throughput bottlenecks", or "indexing efficiency".

•**Implementation:** This typically involves a preliminary LLM call before the retrieval step. LangChain's `MultiQueryRetriever` is a standard implementation that automates this process, generating multiple variations of the query to broaden the search scope.

•**Considerations:** While potentially helpful, overly aggressive expansion can lead to topic drift, where the expanded query retrieves irrelevant documents. Careful prompt engineering and potentially limiting the

Numerical problems on Hybrid Search

Problem 1: Weighted Score Combination

A query is searched against two documents using Hybrid Search.

- BM25 scores:
- Doc1 = 2.5
- Doc2 = 1.8
- Cosine similarity (dense embeddings):
- Doc1 = 0.72
- Doc2 = 0.85

Hybrid score is computed as:

where .

Solution:

- Doc1 =
- Doc2 =

Result: Doc1 is ranked higher.

Problem 2: Normalization + Hybrid Ranking

Query is searched.

- BM25 scores (raw): DocA = 12, DocB = 8
- Cosine similarity: DocA = 0.65, DocB = 0.80

Before combining, BM25 scores are normalized to [0,1]:

Solution:

- For BM25: max = 12, min = 8
- DocA =
- DocB =
- Hybrid score ():
- DocA =
- DocB =

Result: DocA ranked higher.

Problem 3: Top-k Retrieval

Query "network protocols" is searched in a corpus.

- **BM25 scores**: [Doc1=3.2, Doc2=2.9, Doc3=1.5]
- **Cosine similarity**: [Doc1=0.70, Doc2=0.65, Doc3=0.90]
- Weighting: $\alpha = 0.4$.

Solution:
Hybrid scores:

- Doc1 = $0.4 \cdot 3.2 + 0.6 \cdot 0.70 = 1.28 + 0.42 = 1.70$
- Doc2 = $0.4 \cdot 2.9 + 0.6 \cdot 0.65 = 1.16 + 0.39 = 1.55$
- Doc3 = $0.4 \cdot 1.5 + 0.6 \cdot 0.90 = 0.60 + 0.54 = 1.14$

Top-2 documents retrieved: Doc1 and Doc2.

Fine-Tuning LLMs

- Fine-tuning **large language models (LLMs)** is important for tailoring these advanced algorithms to specific tasks or domains.
- This process enhances the model's performance on specialized tasks and significantly broadens its applicability across various fields.
- This means we can take advantage of the natural language processing capacity of pre-trained and **open-source LLMs** and further train them to perform our specific tasks.
- In this tutorial, I'll explain the concept of pre-trained language models and guide you through the step-by-step fine-tuning process, using GPT-2 with Hugging Face as an example.

Understanding How Pre-trained Language Models Work

Image by Author. LLM input and output.

- The Language Model is a type of machine learning algorithm designed to forecast the subsequent word in a sentence, drawing from its preceding segments.
- It is based on the Transformers architecture, which is deeply explained in our article about **How Transformers work.**
- Pre-trained language models, such as GPT (Generative Pre-trained Transformer), are trained on vast amounts of text data.
- This enables LLMs to grasp the fundamental principles governing the use of words and their arrangement in the natural language.

The most important part is that these models are not only good at understanding natural language but are also good at generating human-like text based on the input they receive.

And the best part of it all?

These models are already open to the masses using APIs. If you want to learn how to take advantage of OpenAI's most powerful LLMs, you can learn how to do it by following this cheat sheet on **OpenAI's API.**

- **What is Fine-tuning, and Why is it Important?**
- Fine-tuning is the process of taking a pre-trained model and further training it on a domain-specific dataset.
- Most LLM models today have a very good global performance but fail in specific task-oriented problems. The fine-tuning process offers considerable advantages, including lowered computation expenses and the ability to leverage cutting-edge models without the necessity of building one from the ground up.
- Transformers grant access to an extensive collection of pre-trained models suited for various tasks. Fine-tuning these models is a crucial step for improving the model's ability to perform specific tasks, such as sentiment analysis, question answering, or document summarization, with higher accuracy.

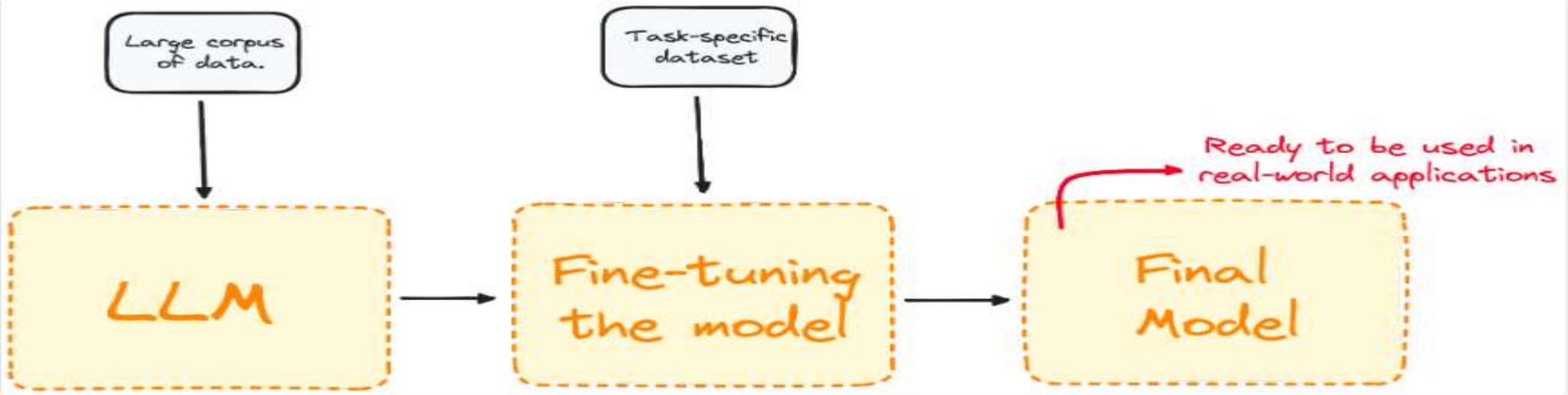


Image by Author. Visualizing the Fine-Tuning process.

Fine-tuning tailors the model to have a better performance for specific tasks, making it more effective and versatile in real-world applications. This process is essential for tailoring an existing model to a particular task or domain.

Whether to engage in fine-tuning hinges on your goals, which typically vary based on the specific domain or task at hand.

The Different Types of Fine-tuning

Fine-tuning can be approached in several ways, depending mainly on its main focus and specific goals.

Supervised fine-tuning

The most straightforward and common fine-tuning approach. The model is further trained on a labeled dataset specific to the target task to perform, such as text classification or named entity recognition.

For instance, we would train our model on a dataset containing text samples labeled with their corresponding sentiment for sentiment analysis.

Few-shot learning

There are some cases where collecting a large labeled dataset is impractical. Few-shot learning tries to address this by providing a few examples (or shots) of the required task at the beginning of the input prompts. This helps the model have a better context of the task without an extensive fine-tuning process.

Transfer learning

Even though all fine-tuning techniques are a form of transfer learning, this category is specifically aimed to allow a model to perform a task different from the task it was initially trained on. The main idea is to leverage the knowledge the model has gained from a large, general dataset and apply it to a more specific or related task.

Domain-specific fine-tuning

This type of fine-tuning tries to adapt the model to understand and generate text that is specific to a particular domain or industry. The model is fine-tuned on a dataset composed of text from the target domain to improve its context and knowledge of domain-specific tasks.

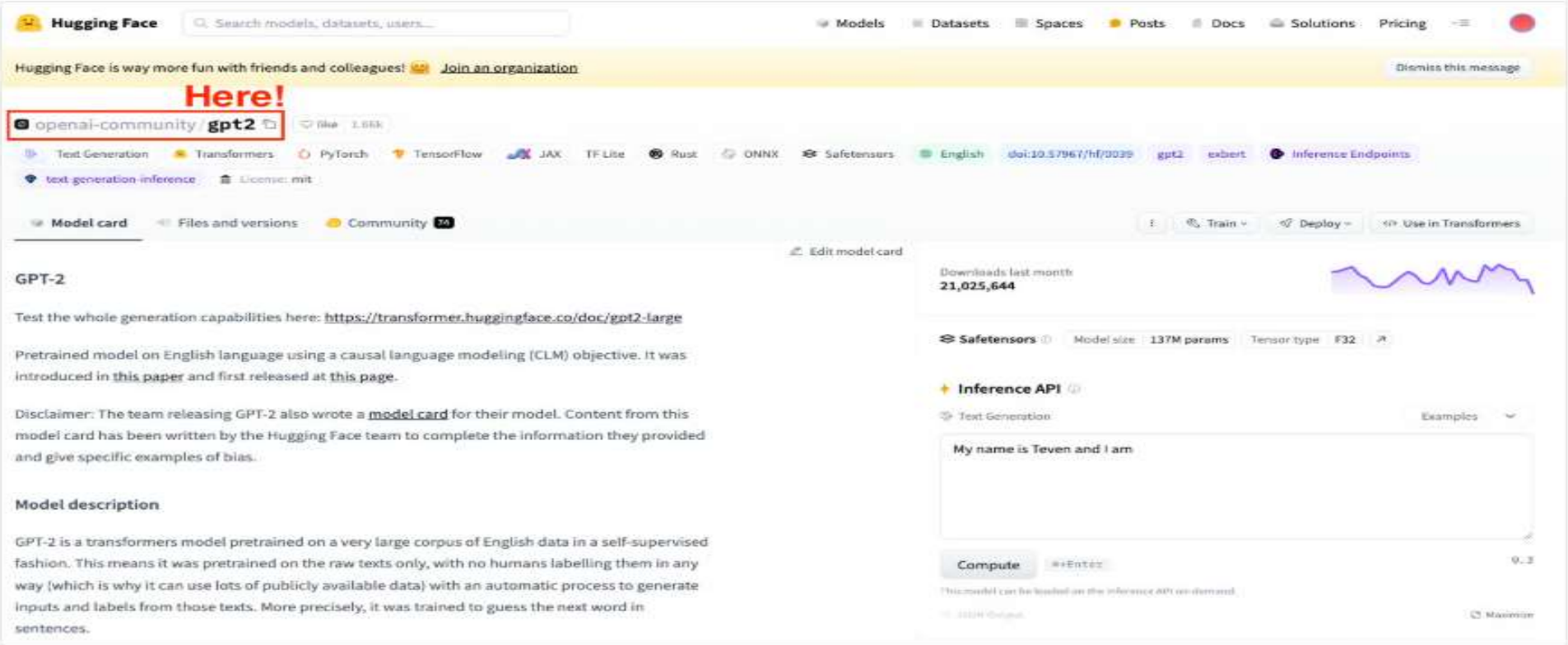
For instance, to generate a chatbot for a medical app, the model would be trained with medical records, to adapt its language understanding capabilities to the health field.

A Step-by-Step Guide to Fine-tuning a LLM

- We already know that Fine-tuning is the process of taking a pre-trained model and updating its parameters by training on a dataset specific to your task. So, let’s exemplify this concept by fine-tuning a real model.
- Imagine we are working with GPT-2, but we detect it is quite bad at inferring the sentiments of tweets.
- One natural question that comes to mind is: Can we do something to improve its performance?
- We can take advantage of fine-tuning by training our pre-trained GPT-2 model from the Hugging Face model with a dataset containing tweets and their corresponding sentiments so the performance improves. Here's a basic example of fine-tuning a model for sequence classification:

Step 1: Choose a pre-trained model and a dataset

To fine-tune a model, we always need to have a pre-trained model in mind. In our case, we are going to perform some simple fine-tuning using GPT-2.



Step 2: Load the data to use

Now that we have our model, we need some good-quality data to work with, and this is precisely where the `datasets` library kicks in.

In my case, I will use the Hugging Face datasets library to import a dataset containing tweets segmented by their sentiment (Positive, Neutral or Negative).

```
from datasets import load_dataset

dataset = load_dataset("mteb/tweet_sentiment_extraction")
df = pd.DataFrame(dataset['train'])
```

Explain code Run code POWERED BY datalab

If we check the dataset we just downloaded, it is a dataset containing a subset for training and a subset for testing. If we convert the training subset to a dataframe, it looks as follows.

	id	text	label	label_text
0	cb774db0d1	I`d have responded, if I were going	1	neutral
1	549e992a42	Sooo SAD I will miss you here in San Diego!!!	0	negative
2	088c60f138	my boss is bullying me...	0	negative
3	9642c003ef	what interview! leave me alone	0	negative
4	358bd9e861	Sons of ****, why couldn`t they put them on t...	0	negative

The data set to be used.

Step 3: Tokenizer

Now that we already have our dataset, we need a tokenizer to prepare it to be parsed by our model.

As LLMs work with tokens, we require a tokenizer to process the dataset. To process your dataset in one step, use the Datasets map method to apply a preprocessing function over the entire dataset.

This is why the second step is to load a pre-trained Tokenizer and tokenize our dataset so it can be used for fine-tuning.

```
from transformers import GPT2Tokenizer

# Loading the dataset to train our model
dataset = load_dataset("mteb/tweet_sentiment_extraction")

tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
tokenizer.pad_token = tokenizer.eos_token
def tokenize_function(examples):
   return tokenizer(examples["text"], padding="max_length", truncation=True)

tokenized_datasets = dataset.map(tokenize_function, batched=True)
```

Step 4: Initialize our base model

Start by loading your model and specify the number of expected labels. From the Tweet's sentiment dataset card, you know there are three labels:

```
from transformers import GPT2ForSequenceClassification

model = GPT2ForSequenceClassification.from_pretrained("gpt2", num_labels=3)
```

Step 5: Evaluate method

Transformers provides a Trainer class optimized for training. However, this method does not include how to evaluate the model. This is why, before starting our training, we will need to pass Trainer a function to evaluate our model performance.

```
import evaluate

metric = evaluate.load("accuracy")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    return metric.compute(predictions=predictions, references=labels)
```

Step 6: Fine-tune using the Trainer Method

Our final step is to set up the training arguments and start the training process. The Transformers library contains the Trainer class, which supports a wide range of training options and features such as logging, gradient accumulation, and mixed precision. We first define the training arguments together with the evaluation strategy. Once everything is defined, we can easily train the model simply using the `train()` command.

```
from transformers import TrainingArguments, Trainer

training_args = TrainingArguments(
    output_dir="test_trainer",
    #evaluation_strategy="epoch",
    per_device_train_batch_size=1,  # Reduce batch size here
    per_device_eval_batch_size=1,    # Optionally, reduce for evaluation as well
    gradient_accumulation_steps=4
    )


trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=small_train_dataset,
    eval_dataset=small_eval_dataset,
    compute_metrics=compute_metrics,

)

trainer.train()
```

Fine-Tuning Large Language Models (LLMs) Using QLoRA

Fine-tuning large language models (LLMs) is used for adapting LLM's to specific tasks, improving their accuracy and making them more efficient.

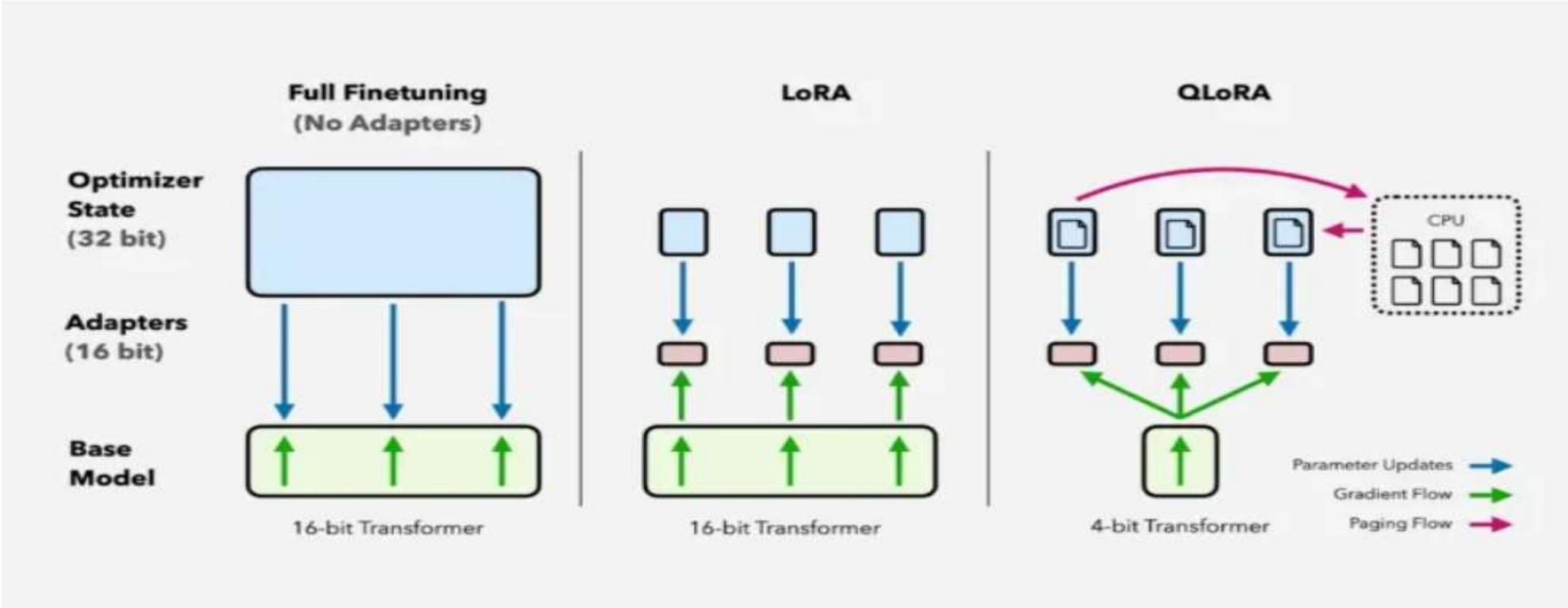
However full fine-tuning of LLMs can be computationally expensive and memory-intensive.

QLoRA (Quantized Low-Rank Adapters) is a technique used to significantly reduces the computational cost while maintaining model quality.

What is QLoRA?

QLoRA is a advanced fine-tuning method that quantizes LLMs to reduce memory usage and applies Low-Rank Adaptation (LoRA) to train a subset of model parameters. This allows:

- **Lower GPU memory requirements** : Fine-tuning large models on consumer GPUs.
- **Faster training** : Using fewer parameters speeds up the process.
- **Preserved model quality** : Achieves similar performance to full fine-tuning.


Full Finetuning
(No Adapters)
LoRA
QLoRA
Optimizer State
(32 bit)
Adapters
(16 bit)
Base Model
CPU
16-bit Transformer
16-bit Transformer
4-bit Transformer
Parameter Updates
Gradient Flow
Paging Flow

Key Components of QLoRA

1.4-bit Quantization (NF4): QLoRA uses Normalized Float 4-bit (NF4) quantization which is optimized for deep learning. Unlike traditional quantization techniques that may introduce numerical instability, NF4 maintains precision by normalizing values in a way that aligns well with deep neural networks.

2.LoRA Adapters: Instead of modifying the full model, LoRA introduces small low-rank matrices into specific layers allowing efficient adaptation with fewer parameters. These adapters fine-tune only critical layers such as query and value projections in transformer models. These layers are chosen because they play a central role in attention mechanisms making fine-tuning more effective without modifying the entire model.

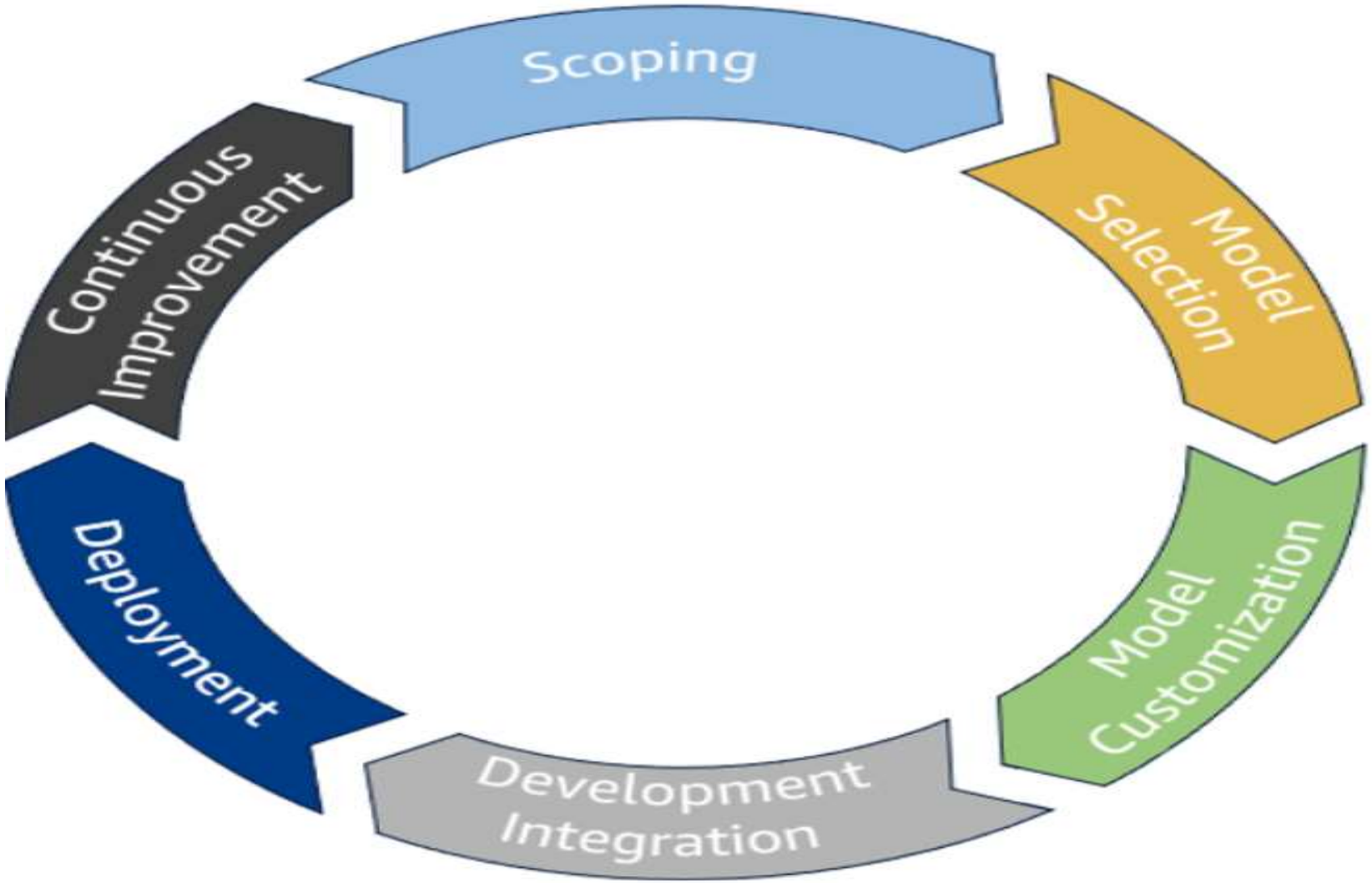
3.Memory: Efficient Training: By combining quantization with LoRA, QLoRA significantly reduces VRAM usage making fine-tuning feasible on consumer-grade GPUs. It achieves this by minimizing activation storage, reducing gradient computation and enabling large-scale training on limited hardware.

Generative AI lifecycle

- The generative AI lifecycle consists of seven key phases: scoping, model selection, model customization, development and integration, deployment, and continuous improvement.

- Each phase of the generative AI lifecycle is evaluated against the six pillars of the Well-Architected Framework.

- This process helps verify that workloads are built and maintained according to best practices across critical aspects of system design and operation.


Scoping
Model Selection
Model Customization
Development Integration
Deployment
Continuous Improvement

Phases of the Generative AI Life Cycle

1. Scoping & Problem Definition

- Identify the business problem and potential use cases.
- Define project goals, feasibility, and expected outcomes.
- Align stakeholders to ensure clarity of objectives.

2. Data Collection & Preparation

- Gather domain-specific datasets (text, images, code, etc.).
- Clean, preprocess, and annotate data to ensure quality.
- Handle bias, privacy, and compliance considerations.

3. Model Selection

- Choose between foundation models (e.g., GPT, LLaMA, Stable Diffusion) or build custom models.
- Evaluate based on scalability, accuracy, and domain fit.
- Consider open-source vs. proprietary options.

4. Model Customization & Training

- Fine-tune or adapt models using transfer learning or prompt engineering.
- Apply techniques like RAG (Retrieval-Augmented Generation) for domain-specific knowledge.
- Optimize hyperparameters for performance.

5. Development & Integration

- Build applications (chatbots, content generators, curriculum tools).
- Integrate with APIs, databases, or enterprise workflows.
- Ensure security and ethical safeguards.

---

6. Deployment

- Deploy models in production environments (cloud, edge, hybrid).
- Ensure scalability, latency management, and cost optimization.
- Use containerization and orchestration (Docker, Kubernetes).

---

7. Monitoring & Continuous Improvement

- Track performance metrics (accuracy, relevance, hallucination rates).
- Collect user feedback for iterative refinement.
- Retrain or update models with new data.
- Ensure compliance with evolving AI governance standards.

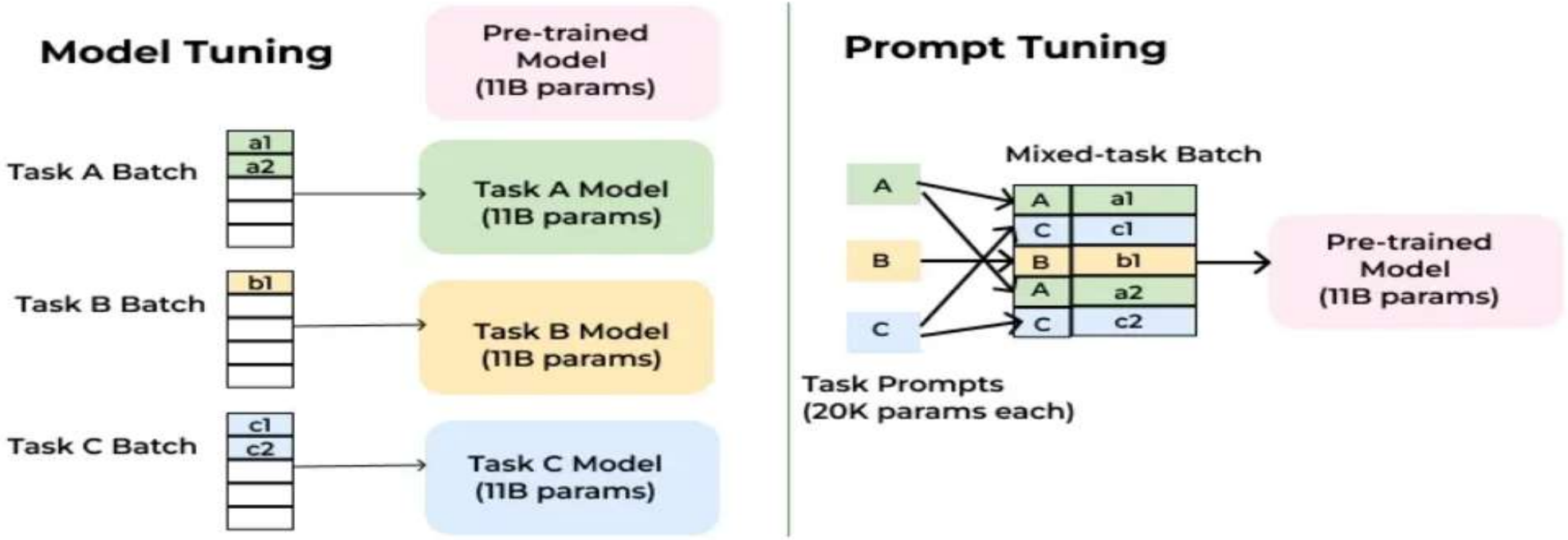
What is Prompt Tuning?

Prompt tuning is a technique that involves modifying the input to a pre-trained language model rather than altering the model's parameters. Instead of fine-tuning the entire model, prompt tuning focuses on designing task-specific "prompts" or instructions that guide the model to produce the desired output.

*A **prompt** is essentially a structured input that provides context or instructions to the model. For example, instead of directly asking the model to classify a sentence as positive or negative, you might prepend the sentence with a prompt like: **"Classify the sentiment of the following sentence: [sentence]."***

Prompt tuning involves freezing the pre-trained model's parameters and optimizing only a small set of additional parameters, often referred to as "soft prompts." These soft prompts are learned during training and serve as a bridge between the model's pre-trained knowledge and the specific task at hand.

How Does Prompt Tuning Work?



1.Pre-trained Model: Start with a pre-trained language model, such as GPT-3 or BERT. The model's parameters remain frozen during the process.

2.Task-Specific Prompts: Design a prompt template that includes placeholders for the input data and task-specific instructions. For example:

2. *"Translate the following English sentence into French: [English sentence]."*
3. *"Summarize the following text in one sentence: [text]."*

3.Soft Prompts: Instead of using fixed, hand-crafted prompts, prompt tuning introduces **learnable soft prompts** . These are continuous vectors that are optimized during training to guide the model toward the desired behavior. Soft prompts are typically initialized randomly and then fine-tuned using gradient descent.

4.Training: During training, only the soft prompts are updated, while the rest of the model remains unchanged. This significantly reduces the computational cost compared to full fine-tuning.

5.Inference: At inference time, the learned soft prompts are prepended to the input, and the model generates predictions based on the combined input.

Advantages of Prompt Tuning

1.Efficiency : One of the most significant advantages of prompt tuning is its efficiency. Since only a small number of parameters (the soft prompts) are updated, the computational and memory requirements are much lower than those of full fine-tuning. This makes prompt tuning particularly appealing for large models with billions of parameters.

2.Scalability : Prompt tuning scales well with model size. Research has shown that as the size of the pre-trained model increases, the performance gap between prompt tuning and full fine-tuning narrows. In some cases, prompt tuning can achieve comparable or even better results than full fine-tuning, especially for very large models.

3.Task Generalization : By leveraging the pre-trained model's knowledge, prompt tuning can generalize better across tasks. The model retains its original capabilities while adapting to new tasks through the use of prompts.

4.Reduced Overfitting : Since prompt tuning modifies only a small subset of parameters, it is less prone to overfitting, especially when working with limited training data.

5.Flexibility : Prompt tuning allows for rapid experimentation with different tasks and prompts without the need to retrain the entire model. This flexibility is particularly valuable in scenarios where multiple tasks need to be addressed simultaneously.

Applications of Prompt Tuning

Prompt tuning has found applications across a wide range of NLP tasks, including:

1.Text Classification : Prompt tuning can be used to classify text into categories such as sentiment, topic, or intent.

2.Question Answering : By providing prompts that guide the model to extract relevant information, prompt tuning can improve performance on question-answering tasks.

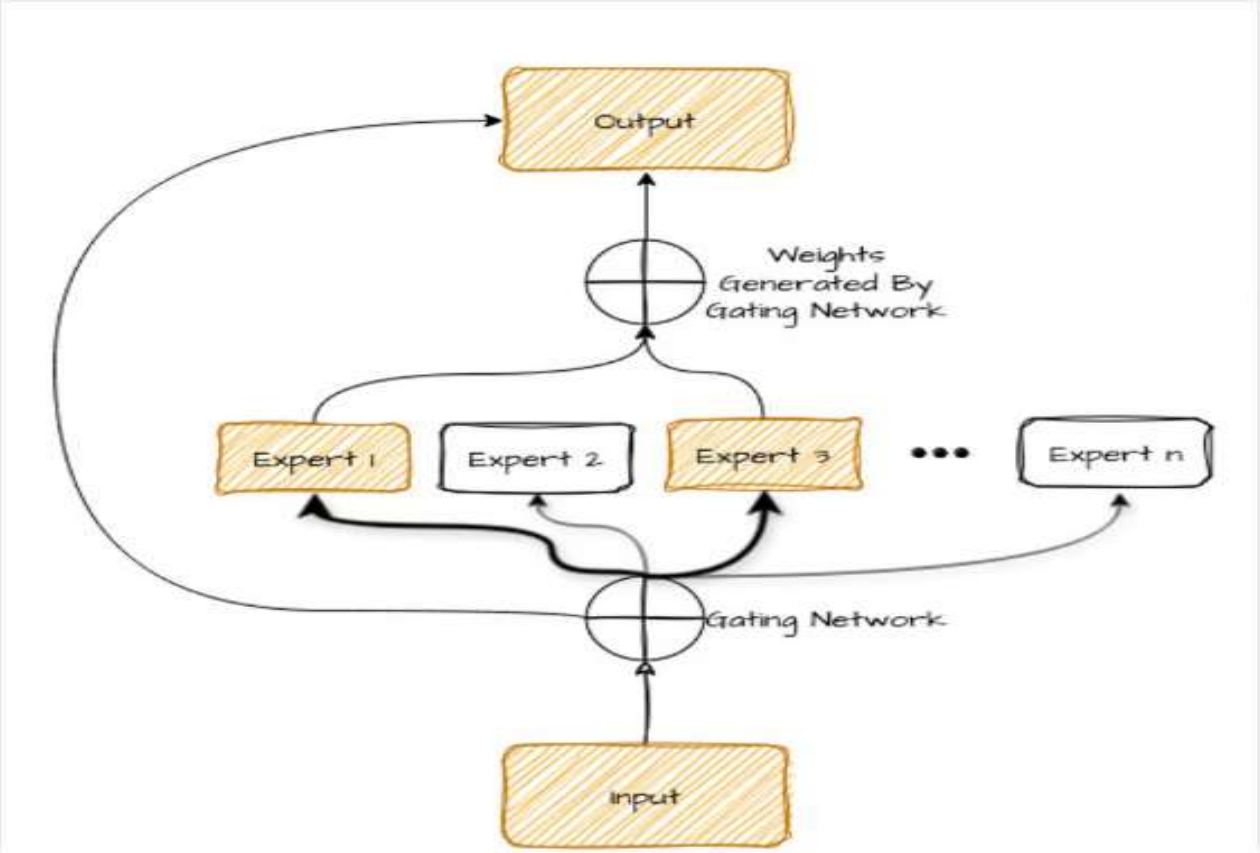
3.Machine Translation : Prompt tuning can help adapt pre-trained models for translation tasks by incorporating task-specific instructions.

4.Summarization : Prompt tuning can be used to generate concise summaries of long documents by guiding the model to focus on key points.

5.Few-Shot Learning : Prompt tuning is particularly effective in few-shot learning scenarios, where the model is given only a small number of examples for a new task. By leveraging the pre-trained model's knowledge, prompt tuning can achieve strong performance even with limited data.

What Is Mixture of Experts (MoE)?

- Imagine an AI model as a team of specialists, each with their own unique expertise. A mixture of experts (MoE) model operates on this principle by dividing a complex task among smaller, specialized networks known as “experts.”
- Each expert focuses on a specific aspect of the problem, enabling the model to address the task more efficiently and accurately. It’s similar to having a doctor for medical issues, a mechanic for car problems, and a chef for cooking—each expert handles what they do best.
- By collaborating, these specialists can solve a broader range of problems more effectively than a single generalist.
- Let’s take a look at the diagram below—we’ll explain it shortly after.


Output
Weights
Generated By
Gating Network
Expert 1
Expert 2
Expert 3
Expert n
Gating Network
Input

Why Evaluating Gen AI Hallucinations, Bias and Toxicity Matters

Generative AI (GenAI) has moved beyond experimental labs into mission-critical enterprise applications, customer interactions, healthcare decision-making, and even financial services. While traditional AI testing prioritized accuracy, F1-scores, and BLEU metrics, these are no longer sufficient for Large Language Models (LLMs).

Accuracy measures functional correctness but often ignores risks like manipulation, stereotype amplification, or harmful advice. The challenge today lies in evaluating bias, toxicity, and hallucinations in generative AI.

High accuracy can actually be dangerous, as it creates “over-trust,” making users more vulnerable to biased or fabricated outputs.

This guide provides a layered strategy to move beyond functional testing toward a model of real-world safety, fairness, and ethical accountability.

In this blog, we provide a **holistic guide** that goes beyond accuracy, addressing how leaders, QA teams, and AI practitioners can evaluate, detect, and mitigate bias, toxicity, and hallucinations in generative AI systems.

Learn how Techment helps enterprises implement AI-powered quality assurance at scale to mitigate **Hallucinations in Generative AI**.

TL; DR: Summary Box

- **Bias isn't accuracy**: A model can score high on benchmarks yet amplify stereotypes or discriminate in outputs.
- **Toxicity needs detection pipelines**: Automated toxicity detection, human-in-the-loop systems, and red-teaming reduce harmful responses.
- **Hallucinations in Gen AI are costly**: LLMs often "make up facts," demanding mitigation strategies like retrieval-augmented generation (RAG).
- **Responsible evaluation requires layers**: Technical, ethical, and business-level reviews must work together.
- **AI safety frameworks are maturing**: Global organizations like Gartner and Forrester stress explainability, fairness, and accountability for mitigating **Hallucinations in Generative AI**.
- **Actionable practices**: Bias testing suites, toxicity filters, fact-checking mechanisms, and domain-specific guardrails are now table stakes.

Discover how low-code test automation can boost QA productivity, cut scripting effort, and speed up releases using Tricentis-powered solutions in our latest blog.

Why Accuracy Metrics Fail for Generative AI

Traditional metrics—accuracy, F1-score, BLEU—capture functional correctness but ignore risks like manipulation, stereotype amplification, or harmful advice. Expert insight extends this further: Models with excellent benchmark scores can still:

- Generate persuasive but misleading content
- Reinforce harmful narratives
- Mask deeper structural unfairness

The “Safety Tax” of Gen AI

Ensuring safety, truthfulness, and fairness is not optional; it’s a fundamental requirement.

High accuracy can create **over-trust**, making users more vulnerable to biased or fabricated outputs.

What Causes Hallucinations in Large Language Models

Regulatory & Ethical Pressures

Regulators worldwide (EU AI Act, U.S. AI Bill of Rights, OECD AI Principles) mandate:

- Transparency
- Explainability
- Fairness
- Accountability

Enterprises ignoring these dimensions risk compliance failures, legal challenges, and reputational damage.

Learn how Techment’s QA services are aligned with global test maturity practices.

What Is Generative AI Bias?

Generative AI bias occurs when AI systems reflect, amplify, or even introduce unfairness due to skewed training data, flawed algorithms, or unintended correlations. Such biases can undermine trust, limit adoption, and create ethical and compliance risks in enterprise applications.

Common Sources of Bias

- **Training Data Bias**
- Historical imbalances (e.g., male-dominated tech forums or Western-centric text corpora) influence the AI's output.
- Data gaps may exclude underrepresented groups, reinforcing stereotypes or producing irrelevant results.
- **Algorithmic Bias**
- Reinforcement learning processes and optimization techniques may unintentionally favor one type of response over another.
- Over-optimization for accuracy can ignore fairness constraints, embedding systematic preference.
- **User Interaction Bias**
- Models fine-tuned on user data inherit the behaviors, language patterns, and cultural skew of those interactions.
- Popularity-based reinforcement (upvotes, likes) amplifies mainstream views while muting minority perspectives.

How to Detect Bias in Generative AI Outputs

Bias arises from the data → model pipeline and is often inherited from social and historical inequalities.

Bias as an Inherited Defect

Models reflect skewed training data—overrepresented groups dominate, underrepresented groups suffer. This leads to:

- Gender-skewed hiring recommendations
- Unequal lending decisions
- Healthcare misdiagnosis

Measuring Bias with True Fairness Metrics

Beyond traditional benchmarking, enterprises must evaluate fairness using:

- **Demographic Parity**
- **Equal Opportunity**
- **Disparate Impact Analysis**
- **Counterfactual Testing**

These reveal harms that accuracy metrics overlook.

Why Counterfactuals Matter

Modifying only a protected attribute (e.g., gender → male/female) exposes subtle but harmful response variations.

Key Insight:
Bias is not a technical bug—it is a societal risk. Evaluating and mitigating bias ensures equitable, inclusive, and trustworthy AI.

Toxicity Detection in AI Outputs

What Is Toxicity?

- **Definition**: Toxicity in AI refers to harmful, offensive, or unsafe outputs generated by models, including hate speech, harassment, and misinformation.
- **Risks**: Toxic outputs damage user trust, brand reputation, and can even lead to legal or compliance issues in regulated industries.
- **Challenge**: Unlike simple factual errors, toxicity often requires context-aware detection, as the same phrase may be benign in one scenario but offensive in another.

Practical Approaches to Detect & Reduce Toxicity

- **Automated Detection Tools**
- Leverage APIs such as **Perspective API**, or open-source classifiers like **Toxic Comment Classification**.
- Enable real-time monitoring for high-volume applications.
- **Human-in-the-Loop Validation**
- Employ content reviewers for edge cases where nuance or cultural context matters.
- Helps avoid false positives that can arise from over-reliance on automation.
- **Red-Teaming AI Systems**
- Systematically stress-test models with **adversarial prompts** to uncover hidden vulnerabilities.
- Ensures robustness against malicious attempts to trigger toxic behavior.

Actionable Practices

- **Zero-Tolerance Filters**
- Enforce strict policies for hate speech, harassment, and violent content.
- **Context-Aware Moderation Pipelines**
- Layer rule-based systems with ML classifiers for more **granular decision-making**.
- **Continuous Toxicity Scoring**
- Implement ongoing evaluations of production outputs.
- Track performance with toxicity benchmarks to ensure **ethical AI deployment**.

According to Reuters, AI-powered moderation reduced **hate speech visibility by 50%** on leading social platforms.

Related Read: Agentic AI vs Generative AI (2026): What Enterprise Leaders Must Know

What Are Hallucinations in Gen AI?

Hallucinations occur when generative AI outputs plausible-sounding but factually incorrect or fabricated responses.

Types of Hallucinations in Generative AI

1. **Intrinsic Hallucinations**: Errors due to flawed reasoning within the model.
2. **Extrinsic Hallucinations**: Confident statements about nonexistent facts, citations, or events.
3. **Domain-Specific Hallucinations**: High-risk in sectors like medicine, finance, and law.

AI Hallucination Mitigation Strategies

- **Retrieval-Augmented Generation (RAG)**: Grounding responses with external knowledge bases.
- **Fact-Checking Layers**: Cross-verification against trusted APIs or databases.
- **Confidence Scoring**: Signaling uncertainty in outputs.

Explore real-world governance scaling in production environments: AI-Powered Automation: The Competitive Edge in Data Quality Management

Measuring Toxicity in AI-Generated Content

Toxicity exists on a **spectrum**—from explicit hate speech to subtle stereotyping and persuasive framing.

The Spectrum of Toxicity

The most harmful toxic outputs are often not overt but subtly harmful, shaping opinions or reinforcing bias quietly.

How Enterprises Should Detect & Reduce Toxicity and Hallucinations in Generative AI

Automated Detection Tools – Use APIs or classifiers (Perspective API, Detoxify) for real-time scanning.

Human-in-the-Loop Validation – Critical for:

- Cultural nuance
- Contextual meaning
- Avoiding false positives

The Role of RLHF

Reinforcement Learning from Human Feedback improves alignment, but its effectiveness depends on the **diversity, representativeness, and quality** of human reviewers.

Red-Teaming: Beyond Benchmarking

Adversarial stress-testing reveals failure modes not visible in standard accuracy tests.

Actionable Practices

- Zero-tolerance toxicity filters
- Multi-layer moderation pipelines
- Continuous toxicity scoring in production

Stat: AI moderation reduced hate speech visibility by 50% on major platforms (Reuters).

Connect with our resident architects to learn more about our AI and other technologies capabilities.

Learn how Techment can help define your AI vision, prioritize high-value use-cases, and build a practical, ROI-driven roadmap with its AI strategy solution offerings.

Responsible GenAI Evaluation Framework For Removing Hallucinations in Generative AI

A robust evaluation strategy must be **layered** across technical, ethical, and business dimensions.

Technical Layer

- Quantitative metrics for bias, toxicity, hallucination
- Benchmark datasets for content safety
- Long-tail edge case testing

Ethical Layer

- Fairness across demographics
- Representation in different cultures and contexts
- Transparency using Explainable AI (XAI)
- XAI: The Audit Trail for Safety

XAI reveals *why* the model behaves as it does, enabling targeted mitigation and compliance reporting.

Business Layer

- Assessing reputational risk
- Ensuring regulatory compliance
- Balancing innovation with safe deployment

The “Harm Tax”

Bias, toxicity, or hallucinations can directly impact:

- Customer trust
- Brand equity
- Financial and legal consequences

Learn how we help organizations evaluate, optimize, and operationalize AI responsibly through our Gen AI evaluation framework.

Learn how Techment helps enterprises measure AI systems beyond accuracy.

GenAI Evaluation Methods

As Generative AI continues to revolutionise content creation, code generation, and language understanding, evaluating its performance accurately becomes crucial.

This guide delves into the most effective methods to assess generative models, covering both quantitative metrics and qualitative techniques.

Whether you’re comparing text generation quality, image synthesis accuracy, or conversational fluency, this post provides step-by-step formulas, intuitive diagrams, and real-world examples to help you understand and apply evaluation metrics like BLEU, ROUGE, FID, perplexity, human evaluation protocols, and more.

1. Automatic Metrics

· **What it is**: Using mathematical formulas to evaluate outputs without human intervention.

· **Common Metrics**:

o **BLEU Score**: Measures how close the generated text is to a reference text (used in text generation).

o **Perplexity**: Measures how well the model predicts a sequence of words (lower is better).

o **FID (Fréchet Inception Distance)**: Evaluates the quality of generated images.